Design and Functional Verification of a Standalone MAX7219 LED Matrix Hardware Driver in Verilog HDL

[M Usman] | Reg No: [2024-EE-393]

INTRODUCTION, OBJECTIVES & LOGIC FLOW

1. Abstract / Introduction

Peripheral communication protocol execution in modern embedded systems frequently relies on software-driven pin-toggling ("bit-banging") handied by a primary CPU core, This approach introduces massive processing overhead, keeping high-performance processors stuck in repetitive execution delay loops just to manage standard serial clocks.

This project presents a hardware-centric solution: an autonomous SPI Master Controller Core engineered specifically to drive the industry-standard MAX7219 LED Matrix Display Driver fC using Veriiog HDt. By shifting the protocol serislization logic ontirely into dedicated FPGA fabric, 16-bit parallel command words (Address + Data) are streanmed with hardware-level timing precision.

To ensure industrial compatibility, the module is engineered as a decoupled, modular Intellectıal Property (IP) core. While fully verified here via a behavioral simulation testbench, this core architecture is uniquely designed for piug-and-piay integration into a Hardware-Software System-on-Chip (SoC) environment, where it can be mapped to an AXI4-Lite bus to interface directly with an AKM or RISC-V processing core.

2. Project Objectives

- Design an independent SPI Master Core governed by an autonomous, synchronous 4-state Finite State Machine (FSM).
- Implement a clock prescaler to step down a high-frequency system cleck to a reliable transmission speed matching the MAS7219 timing requirements.
- Execute high-speed Parallel-to-Serial Conversion utilizing a left-shifting register to stream 16-bit words down a single wire (spi_din).
- Validate logic stabillty via Functional Verification using a ModelSim testbench engineered to emulate the data delivery and trigger puises of an Soc host processor.

3. Industrial Applications

- Industrial Control Displays: Drives critical status arrays independently, ensuring safety readouts remain functional even if the main application processor lags or encounters errors.
- Automotive Control Clusters: Manages real-time dashboard display matriaes and warning clusters in low-power standby states.
- Medical Diagnostic Monitors: Provides deterministic, clock-precise data streaming to prevent visual latency on patient critical tracking aways.

4. System Working Flow

1. Standby Listening: The FSM sits in STATE_IDLE with the ready flag driven High and the chip-select line (spi_cs) held High ($11) to keep the MAX7219 bus asloep.
2. Data Capture: The testbench (acting as a virtual host CPU) drops a 16-bit packet onto tx_data and pulses the start wire like a doorbell.
3. Bus Awakening: The core enters STATE_LOAD, dropping spi_zs Low ($0) to alert the peripheral and latching the 16-bit parailal word into the internal shift_reg.
4. Frequency Division: The system enters STATE_TRANSMIT. A 4-bit counter increments to cleanly divide the master clock frequency.
5. Blt Streaming: The highest bit (shift_reg[1S]) is pushed to the spi_din pin on the rising edgs of the modified clock (spi_cib).
6. Begizter Shift: On the falling edge of the modified clock, the register slides left, updates a bit counter, and prepares the next bit.
7. Display Commit: Once all 16 bits are sent, spi_cs snaps High ($1) in STATE_LATCH to trigger the internal iatch of the MAS7219, locking the pattern into display memory, selely returning the core back to standby.

COLUMN 2: HARDWARE DESCRIPTION LANGUAGE (HDL) CODE

Design Code

```
module led_matrix_driver (
  input  wire      clk,     // System Clock from your testbench
  input  wire      rst_n,   // Active-low Reset
  input  wire [15:0] tx_data,  // The 16-bit packet (Address + Data)
  input  wire      start,   // Trigger signal to kick off transmission

  output reg       ready,   // High when resting, Low when busy
  output reg       spi_clk,  // Clock line to the matrix
  output reg       spi_cs,  // Chip Select line (Active Low)
  output reg       spi_din  // Serial Data line
);
  // State Machine States
  localparam STATE_IDLE    = 2'b00;
  localparam STATE_LOAD    = 2'b01;
  localparam STATE_TRANSMIT  = 2'b10;
  localparam STATE_LATCH    = 2'b11;
  reg [1:0]  current_state, next_state;
  reg [15:0] shift_reg;
  reg [4:0]  bit_counter;  // Counts up to 16 bits
  reg [3:0]  clk_divider;  // Small divider to make the SPI clock visible in
simulation
  // 1. State Transitions
  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) current_state <= STATE_IDLE;
    else      current_state <= next_state;
  end
  // 2. Next State Logic Checklist
  always @(*) begin
    case (current_state)
      STATE_IDLE:   next_state = (start) ? STATE_LOAD : STATE_IDLE;
      STATE_LOAD:   next_state = STATE_TRANSMIT;
      STATE_TRANSMIT: next_state = (bit_counter == 16 && clk_divider
== 4'd15) ? STATE_LATCH : STATE_TRANSMIT;
      STATE_LATCH:  next_state = STATE_IDLE;
      default:      next_state = STATE_IDLE;
    endcase
  end
  // 3. Shifting and Pin Output Logic
  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      ready <= 1'b1; spi_cs <= 1'b1; spi_clk <= 1'b0; spi_din <= 1'b0;
      bit_counter <= 0; clk_divider <= 0;
    end else begin
      case (current_state)
        STATE_IDLE: begin
          ready  <= 1'b1;
          spi_cs  <= 1'b1;
          spi_clk <= 1'b0;
          bit_counter <= 0;
        end
        STATE_LOAD: begin
          ready    <= 1'b0;
          spi_cs   <= 1'b0;    // Pull CS Low to alert the matrix
          shift_reg  <= tx_data;  // Load the 16-bit packet into shift
register
          bit_counter <= 0;
          clk_divider <= 0;
        end

        STATE_TRANSMIT: begin
          clk_divider <= clk_divider + 1;
          spi_din   <= shift_reg[15]; // Always send out the leftmost bit

          // Generate SPI Clock toggle based on our divider
          if (clk_divider == 4'd7) begin
            spi_clk <= 1'b1;    // Rising edge
          end else if (clk_divider == 4'd15) begin
            spi_clk  <= 1'b0;   // Falling edge
            shift_reg <= {shift_reg[14:0], 1'b0}; // Slide bits LEFT
            bit_counter <= bit_counter + 1; // Increment bit count
            clk_divider <= 0;
          end
        end
        STATE_LATCH: begin
          spi_cs <= 1'b1;       // Pull CS High to lock data into the display
          ready  <= 1'b0;
        end
      endcase
    end
  end
endmodule
```

Testbench Code

```
`timescale 1ns / 1ps

module led_matrix_driver_tb;

  // Inputs to the driver (turn into registers in the testbench)
  reg      clk;
  reg      rst_n;
  reg  [15:0] tx_data;
  reg      start;

  // Outputs from the driver (turn into wires in the testbench)
  wire     ready;
  wire     spi_clk;
  wire     spi_cs;
  wire     spi_din;

  // Instantiate (plug in) the Matrix Driver module
  led_matrix_driver uut (
    .clk(clk),
    .rst_n(rst_n),
    .tx_data(tx_data),
    .start(start),
    .ready(ready),
    .spi_clk(spi_clk),
    .spi_cs(spi_cs),
    .spi_din(spi_din)
  );
  // Generate a steady clock signal (toggles every 10ns = 50MHz clock)
  always #10 clk = ~clk;
  initial begin
    // Step 1: Initialize everything
    clk = 0;
    rst_n = 0;
    tx_data = 16'b0455;
    start = 0;

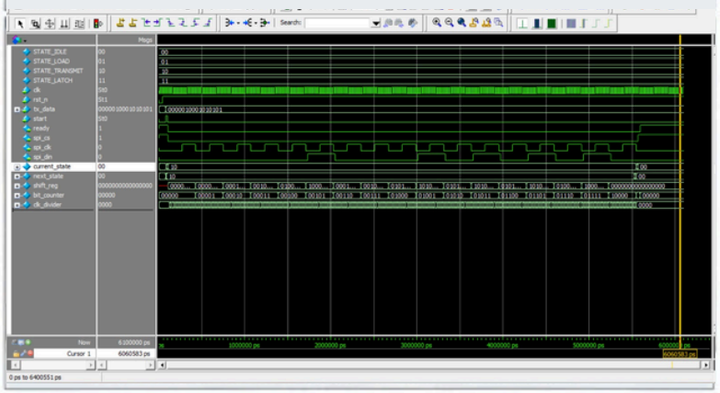
    // Step 2: Hold Reset for a brief moment, then release it
    #40;
    rst_n = 1;
    #40;

    // Step 3: Load sample packet (Address: Row 4 = 0x04 | Data: Pattern = 0x55)
    // Combined 16-bit binary packet: 0000 0100 0101 0101
    tx_data = 16'h0455;

    // Step 4: Pulse the start trigger "doorbell"
    start = 1;
    #20;
    start = 0; // Clear it so it doesn't re-trigger

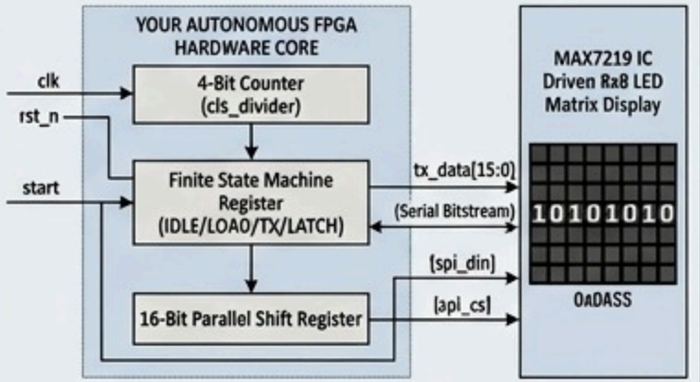
    // Step 5: Wait for the hardware to finish marching out all 16 bits
    // We wait long enough for the transmission loop to execute fully
    #6000;

    // Stop the simulation
    $stop;
  end
endmodule
```



VERIFICATION, KEY FINDINGS, AND FUTURE SCORE

1. Block diagram



2. Functional Verification Waveform Timeline

Simulation Timestamp	Input Stimulus	FSM Vector State	Hardware Pin & Register Logic Execution
0 ns	rst_n = 0	00 (IDLE)	Asynchronous hardware reset. Outputs snap to safe physical defaults
40 ns	rst_n = 1	00 (IDLE)	Standby active. The engine asserts the ready status flag High ($1)
80 ns	start = 1	01 (LOAD)	
100 ns	start = 0	10 (TRANSMIT)	Testbench simulates CPU write hand-shake. Latches 16'h0455; drops spi_cs
5220 ns	Stable	11 (LATCH)	Bit count reaches 16. spi_cs snaps the High to commit data to screen memory
5240 ns	Stable	00 (IDLE)	Core completes task, powers down tracking units, and pulls ready High

3. Key Timing Findings

- Symmetrical Clock Scaling: The modulo-16 synchronous counter (clk_divi-divider) successfully processes the master frequency. By decoding decimal valuos 7 and 15, it stretches a 550 MHz master clock into a into a highly stable 3.125 Mrtz transmission clock.
- 17-Clock Sequential Boundary Delay: The simulation waveform reveals an expected 17-clock wave distribution. This is a characteristic of non-blocking Verliog registers, which require exactly one eatra clock cycle to evaluate the terminal exit condition (bit_counter == 18), ensuring flawless signal clearance.

4. Future Scope: Hardware-Software SoC Integration

- Because the underlying Verilog driver was kept isolated and completely decoupied from any vendor xpesific system architectures, it is 100% Silicon-Ready fnr true System:on-Chip (SoC.) scsling.
- AXI4-Lite Peripheral Wrapping: The next phase of deployment involves wrapping this verified module inside a standard AXI Slave interface wrapper layout, mspping the internal register siv_reg0 directly to the module's 16-bit tx_data input bus.
- Automated Hardware Handshaking: A 1-cycle edge detector circuit will be added to the AXI wrapper to sense whenever the processor writes to the mailbos register, automatically pulsing the hardware start line without requiring softwsre intervention.
- Embedded Software Optimization: This allows an embedded ARM or RISC-V processor to control complex animations on 8x8 matrices via straightforward high-level C software code pointers (XII_Out32[BASE_ADOR, data]), achieving optimal system partitioning and zero CPU stalimg.